Internal review, not an external audit. Source: Blast workspace `audits/2026-11-05-move-deep-audit-4fe8d73-fable/README.md`; its supporting records (system map, function inventory, checked-clean record, proof-of-concept tests and logs) stay in that workspace beside the report. Rendered 5 November 2026 for `blast-v2-contracts/audits/`. Every finding below was fixed in the pull request that added this file; see `audits/README.md`.

# Security Report: Blast launch stack at 4fe8d739

Internal deep adversarial review of the six launch-stack Move packages, written in the format of Asymptotic's October 2026 launchpad report. Reviewed 5 November 2026 by the Fable audit session. Supporting records: system-map.md, function-inventory.md, checked-clean.md; evidence in outputs/verification/2026-11-05-move-deep-audit-4fe8d73-fable.

A concurrent session started an audit of the same commit under the unsuffixed names `audits/2026-11-05-move-deep-audit-4fe8d73/` and `outputs/verification/2026-11-05-move-deep-audit-4fe8d73/`. This record carries the `-fable` suffix so that neither overwrites the other; the owner decides which record to keep or how to merge them.

## Overview

This report covers the launch stack of ADR 0137: `blast_admin` (the shared ACL, the four delayed primitives, and one generic package version and upgrade custody), `blast_core` (basis points, apportionment, recipients and payout, the fee split, the protocol-fee ledger with the partner cut, the treasury, Merkle proofs, the migration hand-off and the owner proof), `blast_vault` (permanent LP custody for every migrated sale, its fee split, Blast's share, the partner hold and holder-reward rounds), `blast_fun` (the bonding-curve launchpad), `blast_presale` (the escrow sale) and `blast_migrators` (the Cetus and Bluefin venues). A coin is sold by one product; when the sale's target is reached the product pays Blast and the named recipients, returns a non-droppable `Migration` carrying the pool's share and the vault's sealed terms, and the next command of the same transaction passes it to the stored venue, which creates the pool, locks or burns the position and shares a vault that holds it for good.

Two immutable packages anchor the stack. `blast_admin` hands every upgradeable package one `Publication` at publish, consumed once by `wrap` into a shared `PackageVersion` (fully paused) and a shared `PackageAdmin` that seals the `Publisher` and `UpgradeCap`; every protected setting is a delayed value whose proposals wait the delay applied when they were scheduled. `blast_core` owns the money vocabulary and the three sealed hand-offs (`VenueTicket`, `Migration`, `OwnerProof`), so products and venues never import each other.

**Key features**

- **Custody stays in the product until migration.** Curve reserves, opening-round pots, presale escrow and every fee balance are `Balance` fields of the shared `Launch` or `Presale`; the only paths out are trades, claims, refunds and the single `migrate`, which empties them exactly (PoC-backed arithmetic in checked-clean.md).
- **One hand-off, one redeemer.** `Migration` has no abilities, carries the stored venue's original type name and terms sealed by the product's own `PackageWitness`, and is redeemable only by that venue; the vault admits a venue and an issuer only from one delayed list and never takes terms from a caller.
- **Delayed administration with a ceiling.** Every setting, action, allowlist and map waits the applied delay, which is at most 30 days; a delay change waits the old delay; upgrade custody starts at 48 hours; `unpause` refuses while any covered delay is zero or a change to zero is pending.
- **Exact apportionment.** Allocation plans, fee splits and presale shares divide totals by largest remainder with Blast's part rounded up and the sum preserved (300-vector property PoC).
- **Fixed supply, no mint authority after creation.** The launchpad mints exactly one billion whole tokens into a ticket and `finalize` seals the supply; the presale requires an unregulated currency with a zero-supply cap and seals the supply the same way.

**Notable specifics**

- **Version gates by enumeration.** Every state-mutating public function of the four upgradeable packages calls `assert_current` before any other guard, with two exceptions: `vault::share_with_position`, reachable only in the transaction of the gated `vault::new`, and `blast_vault_package::unpause`, which is this report's one finding above Informational.
- **Compiled fee maximums.** Trade fees 10 %, protocol share of a trade fee 70 %, one-time allocation 10 % per asset, LP shares 30 % per asset, partner cut 80 %, creation fee 10 SUI, sell burn 50 %, presale raise and token fees 10 %: each enforced where the value is built and, in the presale, again where it is consumed.
- **Regulated Quotes are admitted by design.** USDC on mainnet carries a deny cap; a deny-listed recipient can block a payout, a vault's collection and its re-split (accepted in the ADR 0134 amendments). Sale tokens are never regulated.
- **The migrators package has no pause bits.** Its collection and preparation calls stop only when an upgrade advances the version (accepted residual, ADR 0137 amendment).
- **Interface stubs match mainnet.** The pinned Cetus CLMM v15, Cetus LP Burn v8 and Bluefin Spot v17 signatures were verified against the live bytecode on 5 November 2026 by the repository gate and by an independent GraphQL pull.
- **Not yet published.** Every package manifest still names its own address `0x0`; the ADR 0134 stack is a fresh lineage, so the deployment runbook in `deployments/README.md` is a release condition, not a live state.

## Scope

- **Project:** blast-v2-contracts (`interest-protocol/blast-v2-contracts`)
- **Branch:** `main`
- **Audited commit:** `4fe8d739f04bafd0775691795eb9f3727e984089` (PR #212, 5 November 2026 10:39 UTC), read from an isolated `git archive` snapshot. The stale local checkout and its uncommitted `Move.lock` edits were not used.

| In-scope package | Path |
|---|---|
| `blast_admin` | `sui/admin` |
| `blast_core` | `sui/core` |

| In-scope package | Path |
|---|---|
| `blast_vault` | `sui/vault` |
| `blast_fun` | `sui/launchpad` |
| `blast_presale` | `sui/presale` |
| `blast_migrators` | `sui/migrators` |

**In scope**

- `blast_admin`: the ACL and root transfer, the four delayed primitives, `Publication` → `wrap`, pause bits, scheduled upgrades and immutability, Publisher borrowing.
- `blast_core`: basis points and apportionment, recipients and pay-or-vest, fee splits, the protocol-fee ledger and partner payout, the treasury, Merkle verification, the hand-off and owner proof, the seed-price math.
- `blast_vault`: the witness registry and `unpause`, vault creation and position custody, fee splitting and routing, auto-LP and buyback receipts, owner claims and split changes, Blast's per-vault share, holder-reward scheduling, publication, claims, gas settlement and closure.
- `blast_fun`: launch tickets, intake and finalization, the curve (buys, sells, the capped crossing), opening rounds, creator authority and cap reassignment, metadata, claims, migration and the owner proof, launch and runtime configuration, `unpause`.
- `blast_presale`: creation and settings, rate changes, the three commitment paths with caps, phases and hard caps, settlement and early close, cancellation, claims and vesting, migration and the owner proof, creation and fee policies, `unpause`.
- `blast_migrators`: Cetus and Bluefin preparation, migration, collection, settled split updates, buybacks, the venue configuration, and the fidelity of every external function the two venues call against mainnet bytecode.
- The seams: core → vault → venues, launchpad → core → migrators, presale → core → migrators, `blast_admin` delayed modules → every consumer, partner cut → every ledger payout.

**Out of scope**

- `blast_partners`, `blast_sui`, `boosts/*`, `otc`, `vesting`, `coin_template`, `launch_ticket_template`, `presale_testkit`, `vendors/*` and the `interfaces/*` stubs as logic. This report makes no statement about them beyond what Trusted Dependencies relies on.
- The Rust PTB harness and the venue fork gate (`sui/launchpad/ptb_tests`), the SDK, blast-infra decoders, the admin console and off-chain signers.
- Formal verification; this review tests, it does not prove.

**Dependencies:** `blast_partners`, `blast_fun_vesting`, the generated coin packages, the Sui Framework and Move Standard Library at the pinned revision, and the three external venue packages for interface fidelity. Trusted Dependencies sets out what each is trusted for.

**Test suites run** (`sui move test --build-env mainnet --threads 1`, CI fixture addresses, one package per heavy slot): admin 198/198, core 79/79, vault 120/120, launchpad 444/444, presale 282/282, migrators 40/40 (1,163 stock tests). PoC tests written for this audit: core 3/3, vault 3/3, launchpad 3/3. Repository gates: `check_move_style.py` (233 modules), `check_move_errors.py` (337 errors with exact-abort oracles), `check_admin_delay.py`, every package's `check_source_boundaries.sh`, `check_mainnet_abi.mjs` self-test 74/74 and live PASS.

## Summary of Results

| Severity | Confirmed findings |
|---|---|
| Critical | 0 |
| High | 0 |
| Medium | 0 |
| Low | 1 |
| Informational | 4 |

Hypotheses (not counted above): 1.

No finding lets an untrusted party take, redirect or permanently lock funds. The Low concerns upgrade hygiene: the vault's `unpause` wrapper is the only state-changing public function in the four upgradeable packages that skips the version gate, so after a future upgrade an `UnpauseAdmin` could release the vault through superseded bytecode that enforces an older, weaker gate. The Informational findings concern a release gate that does not reach the core treasury's delay, an event that omits the vesting schedule its presale twin carries, holder-reward payouts that strand at object addresses named by the off-chain manifest, and the coupling of auto-LP compounding to permissionless fee collection. Everything attacked under the §4 catalogue and found to hold is recorded in checked-clean.md.

## Findings

| ID | Title | Severity | Status |
|---|---|---|---|
| L-1 | The vault's `unpause` is not version-gated, so superseded bytecode can release pause bits after an upgrade | Low | Open |
| I-1 | The core `Treasury` delay is outside every release gate; one visible 48-hour wait turns redirection into a one-call setter | Informational | Open |
| I-2 | `MigrationAllocationPaid` omits the vesting schedule ID that the presale's allocation events carry | Informational | Open |
| I-3 | Holder-reward claims are relayable transfers to the leaf's beneficiary, so a leaf naming an object address strands its payout | Informational | Open |
| I-4 | Auto-LP compounding runs inside permissionless fee collection, so any venue-side abort in the add blocks that vault's collection | Informational | Open |

### L-1 · LOW · Open

**The vault's `unpause` is not version-gated, so superseded bytecode can release pause bits after an upgrade**

**Location:** `blast_vault`, `sui/vault/sources/blast_vault_package.move:148-158`; compare `sui/launchpad/sources/launch_config.move:383-399`, `sui/presale/sources/creation_policy.move:180-196` and the vault's own `make_immutable` at `blast_vault_package.move:163-172`.

**Description.** `blast_vault_package::unpause` checks `self.witnesses.is_guarded()` and then calls the generic `package_admin::clear_pause_bits`, which by design reads no consumer version (`sui/admin/sources/package_admin.move:254-269`). Neither call compares the shared `PackageVersion<BLAST_VAULT_PACKAGE>.version` with the bytecode's compiled `constants::version!()`. The launchpad's and the presale's `unpause` wrappers, and the vault's `make_immutable` wrapper two functions below, all begin with `package_version.assert_current(constants::version!())`. On Sui a superseded package version stays callable and shares every type with its successors, so once `commit_upgrade_and_advance` moves the version to 2, version-1 bytecode's `unpause` still accepts the live `PackageVersion`, still passes the version-1 guard, and clears the requested bits.

**Impact.** A future vault release that tightens its release gate, for example by covering a new delayed setting, cannot stop an `UnpauseAdmin<BLAST_VAULT_PACKAGE>` from releasing the vault through the version-1 wrapper, which only checks the witness-admission delay. The role is required, so no untrusted party can trigger it; the package is unpublished and no second version exists, so nothing is exposed today. This is the Asymptotic L-5 class: a guard the package's own convention calls for is missing on one function.

**Recommendation.** Add `package_version.assert_current(constants::version!());` as the first statement of `blast_vault_package::unpause`, matching the product wrappers, and add the test the launchpad and presale already have (`stale_version_is_rejected_before_the_delay_gate`) to `blast_vault_package_tests.move`: set the version to `constants::version!() + 1` and expect `blast_admin::errors::EVersionMismatch` at `blast_admin::package_version`. Body-only change; the signature stays.

**Evidence.** `outputs/verification/2026-11-05-move-deep-audit-4fe8d73-fable/poc/vault/audit_poc_vault_tests.move`. `audit_poc_stale_version_unpause_still_clears_bits` pauses the admin bit, sets the stored version to 2 and clears it through `unpause` from version-1 bytecode; two controls show the same stale bytecode's `schedule_witness_delay` and `make_immutable` abort with `EVersionMismatch`.

```
cd sui/vault && sui move test --build-env mainnet --threads 1 audit_poc
[ PASS    ] blast_vault::audit_poc_vault_tests::audit_poc_stale_version_gated_registry_call_is_rejected
[ PASS    ] blast_vault::audit_poc_vault_tests::audit_poc_stale_version_make_immutable_is_rejected
[ PASS    ] blast_vault::audit_poc_vault_tests::audit_poc_stale_version_unpause_still_clears_bits
Test result: OK. Total tests: 3; passed: 3; failed: 0
```

Log: `logs/poc-vault.log`.

### I-1 · INFORMATIONAL · Open

**The core `Treasury` delay is outside every release gate; one visible 48-hour wait turns redirection into a one-call setter**

**Location:** `blast_core`, `sui/core/sources/blast_treasury.move:32-47`; the gates at `sui/launchpad/sources/launch_config.move:393-396` and `runtime_config.move:173-175`, `sui/presale/sources/creation_policy.move:190-193`, `sui/vault/sources/blast_vault_package.move:155`.

**Description.** The treasury destination is a `Delayed<DestinationKey, address>` compiled at 48 hours, so no setup step can leave it at zero. `schedule_destination` takes the next `delay_ms` from the `TreasuryAdmin`, and `delayed::schedule` accepts any value up to 30 days, zero included; the proposal waits the applied 48 hours. Once applied at zero, every later `schedule_destination` and `apply_destination` pair runs in one transaction. The pre-audit's LP-L1 fix made each product's `unpause` assert that the notices it covers are guarded; at this commit the treasury moved to `blast_core`, and no product's or the vault's gate reads `Treasury::is_guarded`. The ADR 0134 hardening amendment chose "nonzero, not a fixed 48-hour floor, and no permanent floor", so a reduction is a visible, cancellable proposal; nothing in code, however, flags a treasury that has already reached zero.

**Impact.** A `TreasuryAdmin` whose key is compromised needs one uncancelled 48-hour proposal, after which Blast's fees from every launch, presale and vault go to an address it chooses in the same call as the redirect; `claim_protocol_fees`, `pay_protocol_fees_with_partner` and every vault collection read the destination live. `CancelAdmin` can cancel the first proposal, and the monitoring the pre-audit asked for (`DelayedScheduled.delay_ms == 0`) catches it. No untrusted party can trigger anything.

**Recommendation.** Either have `blast_treasury::schedule_destination` reject a zero next delay (a floor only for this one setting, which the owner may decline per ADR 0134), or make the three `unpause` wrappers and the vault's also assert `treasury.is_guarded()`, and add a test that `unpause` refuses while the treasury delay is zero. Record the chosen answer in ADR 0134.

**Evidence.** `poc/launchpad/audit_poc_launchpad_tests.move::audit_poc_release_gate_does_not_cover_the_treasury_delay` lowers the delay to zero through one 48-hour wait, redirects to `0xBEEF` in one call, and shows `launch_config::unpause` still clearing every bit. Log: `logs/poc-launchpad.log`.

### I-2 · INFORMATIONAL · Open

**`MigrationAllocationPaid` omits the vesting schedule ID that the presale's allocation events carry**

**Location:** `blast_fun`, `sui/launchpad/sources/launch.move:858-868` and `sui/launchpad/sources/lib/events.move:198-210,372-378`; compare `sui/presale/sources/events.move:123-144`.

**Description.** ADR 0137 owner decision 6 let launchpad recipients vest. `launch::migrate` calls `payout::pay_or_vest` for each recipient's Meme and Quote and discards the `Option<ID>` it returns, then emits `MigrationAllocationPaid { launch_id, destination, meme_amount, quote_amount }`. The presale's `RaiseAllocated` and `RecipientAllocated` carry `vesting_id: Option<ID>` for the same primitive. A vested recipient's coins leave the launch into one or two shared `Vesting` objects created in the same transaction; the launchpad event says they were paid.

**Impact.** Indexing only. The vesting package emits `VestingCreated { vesting_id, coin_type, beneficiary, total_amount, … }` in the same transaction, so a reducer can recover the fact by joining on transaction, beneficiary and amount; a reducer that reads the launchpad stream alone will show vested recipients as paid. The lineage is unpublished, so the schema can still change at no cost.

**Recommendation.** Add `meme_vesting_id: Option<ID>` and `quote_vesting_id: Option<ID>` to `MigrationAllocationPaid` and emit the two IDs `pay_or_vest` returns, as the presale does; pin them in the existing migration-vesting test (`migration_vests_a_vesting_recipients_allocation`).

**Evidence.** By reading; the emitter signature at `events.move:198-204` takes no schedule ID.

### I-3 · INFORMATIONAL · Open

**Holder-reward claims are relayable transfers to the leaf's beneficiary, so a leaf naming an object address strands its payout**

**Location:** `blast_vault`, `sui/vault/sources/holder_rewards.move:193-243`; `sui/core/sources/payout.move:103-112`.

**Description.** `claim` verifies the leaf `(vault, round, index, beneficiary, meme, quote)`, marks its bit, and pays with `payout::pay`, a `transfer::public_transfer` of a fresh coin to `beneficiary`. Anyone may relay a proof. The round's leaves come from an off-chain allocation of checkpointed holder balances; some of those balances belong to objects, not accounts (DEX pools, vesting schedules, other vaults). A coin transferred to an object's address can be recovered only by that object's module through `transfer::receive`; a Cetus pool or a `Vesting` object has no such path. The claim succeeds, the bit is set, and the Meme and Quote are gone; `close_round` cannot return them because the leaf is marked claimed.

**Impact.** Bounded by the leaf amounts the manifest assigns to object addresses; no third party gains. A griefer who wants to burn those rewards only has to relay the proof before the deadline, which is permitted by design. The control is off chain: the manifest must exclude object-held balances or route them to an account.

**Recommendation.** Document the exclusion in the holder-rewards manifest specification and the `RewardsAdmin` runbook, and have the off-chain checker refuse a leaf whose beneficiary is a known object (any address that resolves to an object at the checkpoint). On chain nothing cheap distinguishes the two.

**Evidence.** By reading: `holder_rewards.move:231-232` transfers unconditionally; `vault.move`/`holder_rewards.move` offer no receive path.

### I-4 · INFORMATIONAL · Open

**Auto-LP compounding runs inside permissionless fee collection, so any venue-side abort in the add blocks that vault's collection**

**Location:** `blast_migrators`, `sui/migrators/sources/cetus.move:320-342,378-441` and `sui/migrators/sources/bluefin.move:300-313,334-361,389-426`.

**Description.** Both venues' `collect_fees` and `settle_and_apply_fee_split` call `settle_fees`, which collects, splits through the vault and then `compound`s: it takes the whole auto-LP balance with a receipt, chooses a side, calls the venue's add-liquidity, and returns the rest. The side choice is meant to guarantee the venue never needs more of the other asset than the vault holds. Cetus's choice calls Cetus's own `clmm_math::get_liquidity_by_amount` with the same inputs the add uses, so it agrees by construction. Bluefin's choice re-derives the venue's formulas locally with a two-atom margin (`can_fix_a`, `can_fix_b`). If the venue ever asks for more than the local estimate plus the margin, `add_liquidity_with_fixed_amount` aborts in the venue, and because compounding is on the collection path, every `collect_fees` and `settle_and_apply_fee_split` on that vault aborts until the auto-LP balance changes, which only a collection can do. The owner's fee split cannot be changed to drop the auto-LP route either, because applying a split also settles first.

**Impact.** No funds move; a vault's fees stay in the position until a migrators upgrade (48-hour notice) adds a path that collects without compounding. The trigger is not demonstrated (see H-1) and the stock suite's grid and extreme-value tests of the Bluefin side selection pass. A Cetus or Bluefin pause has the same effect through `collect_fee` itself and is an accepted venue-operator lever (MG-I1).

**Recommendation.** Keep compounding off the collection path: either make `compound` its own permissionless call, or have `collect_fees` skip the add and leave the balances waiting when the venue's own preview says the add would not fit. Add a test that a collection succeeds when the auto-LP balances cannot be added.

**Evidence.** By reading; the control flow at `cetus.move:332-341` and `bluefin.move:310-312` is unconditional.

### Hypotheses

| ID | Statement | Why it could not be proven |
|---|---|---|
| H-1 | Bluefin's `add_liquidity_with_fixed_amount` can require more of the other asset than `auto_lp_fixed_side` estimates plus two atoms for some pool state, which through I-4 would stop that vault's collection. | The venue's pool bytecode cannot execute locally (the interface is `abort 0` stubs); the Rust fork gate that runs real venue bytecode needs a full simulator build that the host's disk and memory budget did not allow in this session. |

## Verification

This audit tests; it does not prove. For each package, the invariants from the system map and how each is covered: by a stock test (S), by a PoC written for this audit (P), by reading only (R), or not at all (–).

| Package | Invariant | Coverage |
|---|---|---|
| admin | one `Publication` per package, consumed once by `wrap` with a version-1 cap | S |
| admin | version moves only by +1 inside `commit_upgrade_and_advance` | S |
| admin | proposals wait the applied delay; a shorter next delay waits the old one; ≤ 30 days | S |
| admin | `take_action`/`apply_delay` reject the other proposal kind | S |
| admin | unpause needs the package witness and a guarded upgrade delay | S |
| admin | a revoked or destroyed cap signs in to nothing; root transfer waits 7 days | S |
| core | apportionment sums to the total and the rounded-up part gets its ceiling | S, P |
| core | ledger pays only the treasury, partner first when bound, destroyed only empty | S |
| core | hand-offs cannot be dropped; `redeem` requires the stored venue; `new` requires the sealing witness | S |
| core | seed price is the exact floor square root; seedable bounds reciprocal | P, S |
| core | Merkle leaves and nodes are domain-separated; depth and index exact | S, P |
| vault | venue and issuer both admitted; one vault per (issuer, origin); positive liquidity | S |
| vault | collection conserves gross = cut + Σ routes; nothing left over | S |
| vault | receipts bind the vault and cap returns | S |
| vault | rounds: number sequential, interval non-overlapping, budget within rewards, bitmap once, deadline recovery | S |
| vault | owner actions need a proof for the exact terms; 48-hour fixed split and share delays | S |
| vault | every state-mutating function is version-gated | P (L-1 found) |

| Package | Invariant | Coverage |
|---|---|---|
| launchpad | one launch per Meme; pair distinct; Quote, venue, decimals admitted; supply exact; sell burn ≤ 50 % | S |
| launchpad | capped net input equals the gap; `reaches_target` ⇔ `real == target` | P, S |
| launchpad | buy-then-sell and split buys never profit; sells never overdraw the real reserve | P, S |
| launchpad | opening-round pot and bids move together; delivery exact, last takes remainder | S |
| launchpad | `migrate` empties every balance and pays exact shares | S |
| launchpad | creator claim one-time, bound to sender or signed message, caps honoured by ID | S |
| launchpad | release gate covers every launchpad delay | S; treasury not covered (I-1, P) |
| presale | one presale per coin; unregulated, unknown-supply currency; supply sealed | S |
| presale | shares sum exactly; LP share positive; seedable at creation and after rate changes | S |
| presale | commitments respect caps, minimums, participants, hard cap | S |
| presale | claims pro rata, last takes remainder; cancellation refunds all | S |
| presale | `migrate` moves exactly the target and the LP tokens | S |
| migrators | only the stored venue redeems; pair exact; Cetus order canonical | S (unit), R (venue) |
| migrators | Cetus creation reserved before Meme leaves custody; no adoption | R; fork gate not run |
| migrators | collection touches only the vault's pool and routes through `split_fees` | R |
| migrators | auto-LP add never overdraws | S (side selection), – (venue execution; H-1) |

**Coverage matrix** (package × §4 subsection): a finding ID, `Clean` (attacked, held; details in checked-clean.md) or `N/A` with a reason.

| Package | 4.1 Objects | 4.2 PTBs | 4.3 Arithmetic | 4.4 DEX | 4.5 Authority | 4.6 Protocol | 4.7 Bounds | 4.8 Events | 4.9 Tests |
|---|---|---|---|---|---|---|---|---|---|
| admin | Clean | Clean | Clean (deadline math) | N/A: no venue | Clean | N/A: no economics | Clean (roles ≤ 32) | Clean | Clean |
| core | Clean | Clean | Clean | N/A: price math only, no venue call | I-1 | Clean (ledger, Merkle) | Clean (routes ≤ 20) | Clean | Clean |
| vault | Clean | Clean | Clean | Clean (receipts) | L-1 | I-3 | Clean (leaves ≤ 65,536) | Clean | gap: stale-version unpause test |
| launchpad | Clean | Clean | Clean | N/A: no venue call (hand-off only) | Clean | Clean | Clean (recipients ≤ 16) | I-2 | gap: seedable-price pin |
| presale | Clean | Clean | Clean | N/A: hand-off only | Clean | Clean | Clean (participants ≤ 65,536, recipients ≤ 64) | Clean | Clean |
| migrators | Clean | Clean | Clean (Bluefin u256 math) | I-4, H-1; fidelity Clean | Clean (no pause by design) | N/A: no sale logic | Clean | Clean | gap: venue behaviour only in the fork gate |

## Trusted Dependencies

| Dependency | Kind | Address |
|---|---|---|
| `blast_partners` | package (local, immutable) | — (unpublished lineage) |
| `blast_fun_vesting` | package (local, immutable) | — |
| generated coin packages (`launch_ticket_template`, presale creators' coins) | packages | — |
| Sui Framework | framework, rev `06734f6ff0af45d8632a14a4dc4b100197f6b1a2` | `0x0000000000000000000000000000000000000000000000000000000000000002` |
| Move Standard Library | framework, same rev | `0x0000000000000000000000000000000000000000000000000000000000000001` |
| Cetus CLMM v15 | external package | original `0x1eabed72c53feb3805120a081dc15963c204dc8d091542592abaf7a35689b2fb`, current `0x260693ec785a6e6c9d81d58c7d2ff72f1288ae0fa6a9725abe05a6478b11f084` |
| Cetus LP Burn v8 | external package | original `0x12d73de9a6bc3cb658ec9dc0fe7de2662be1cea5c76c092fcc3606048cdbac27`, current `0xc4aabc874f5060d3147d8f74c14e3a8a1f9f8d92edea138a51635395ed167cd9` |
| Bluefin Spot v17 | external package | original `0x3492c874c1e3b3e2984e8c41b589e642d4d0a5d6459e5a9cfc2d52fd7c89c267`, current `0xd075338d105482f1527cbfd363d6413558f184dec36d9138a70261e87f486e9c` |
| `integer_mate` v7 | external package | `0x714a63a0dba6da4f017b42d5d0fb78867f18bcde904868e51d951a5a6f5b7f57` |

The stack's guarantees hold only relative to the dependencies below, which this review takes as correct.

| Dependency | Trusted for |
|---|---|
| `blast_partners` | `Partner` objects are shared and never destroyed, so a bound `partner_id` always resolves; `pay` sends at most the applied share, rounded down, to the cap-set recipient and leaves the rest; shares ≤ 8,000 bps change only after a fixed 7-day delay; `PartnerList` accepts a partner at once and keeps a removed one accepted for that delay |

| Dependency | Trusted for |
|---|---|
| `blast_fun_vesting` | `new_irrevocable` locks the whole coin for the named beneficiary from `start_ms = now` with the given cliff and steps, rejects a zero beneficiary and a past start, and lets only the beneficiary claim vested value; `VestingCreated` carries the schedule ID, beneficiary and amount |
| generated coin packages | the `launch_ticket_template` initializer passes its one-time witness and the creator's choices to `launch_ticket::new` and delivers the ticket and `TreasuryCap` to the publisher; presale creators' coins are registry currencies whose `TreasuryCap` holder is the sale's creator |
| Sui Framework | conservation of `Balance` and `Coin`; Coin Registry supply modes (`make_supply_fixed`, `make_supply_burn_only`, `burn_balance`), metadata caps and the `Currency` BCS layout; dynamic fields and dynamic object fields; derived object IDs (one claim per key); package upgrade tickets, receipts, policies and `make_immutable`; Ed25519 verification; the clock; shared-object deletion; transfers and `public_transfer` deny-list enforcement |
| Move Standard Library | type identities through `type_name` (`with_original_ids`, `with_defining_ids`, `original_id`, `is_primitive`); checked u64/u128/u256 arithmetic, `mul_div`, `mul_div_ceil`, `div_ceil`, `checked_add`; options, vectors, BCS |
| Cetus CLMM v15 | `get_fee_rate` aborts for an unregistered spacing; `mint_pool_creation_cap` mints one cap per coin type from its `TreasuryCap`; `register_permission_pair` makes the pair's creation at that spacing permissioned; `create_pool_v3_with_creation_cap` creates exactly one pool at the given price and seeds the full range from the coins passed, returning the position and residuals; `add_liquidity_fix_coin`/`repay_add_liquidity` pay exactly `add_liquidity_pay_amount`; `flash_swap`/`repay_flash_swap` by amount in; `clmm_math::get_liquidity_by_amount` is the function the add path uses |
| Cetus LP Burn v8 | `burn_lp_v2` permanently locks the position's liquidity and returns a proof that `collect_fee` can use to collect the position's fees and nothing else |
| Bluefin Spot v17 | `create_pool_and_get_object` creates a fresh pool for exactly the configured SUI fee at the given price and tier; `open_position` and `add_liquidity_with_fixed_amount` return the unused balances; `collect_fee` and `swap` as named; the pool manager role is limited to reward emissions (ADR 0088) |
| `integer_mate` | `i32` arithmetic for full-range tick alignment |

Two dependencies carry the most weight. Every role-gated result assumes the ACL hands a role's witness only to a live `AdminCap` holding that role, so access control is exactly as strong as the ACL and the custody of its root. Every venue result assumes the external packages behave as their verified signatures suggest; the fork gate, not this review, is the local evidence for their behaviour.

## Coverage and Limitations

This report establishes properties of the six packages at the audited commit relative to the dependencies above, by source review and local tests. It does not establish the following.

- **Venue execution.** No Cetus or Bluefin bytecode ran in this review; the interface stubs are `abort 0`. Pool creation, seeding, burning, fee collection, auto-LP adds and swaps are covered only by the repository's fork gate at its last recorded run, which this session could not build (the `ptb_tests` simulator needs more disk and memory than the shared host budget allowed on 5 November). H-1 and the venue rows of the verification table depend on it.
- **PTB-level behaviour.** Result handling, `entry` restrictions and multi-command flows were traced by reading, not executed. Every PoC is a Move unit test.
- **Interface fidelity scope.** Signatures, type parameters, abilities, type-argument order and module fingerprints were compared against mainnet; behaviour was not. A venue upgrade after 5 November invalidates the comparison.
- **Upgrades.** Old bytecode stays callable forever; a governed upgrade under the compatible policy can replace any body in the four upgradeable packages after its notice. `blast_admin`, `blast_core` and `blast_partners` are to publish immutable, so a defect in them needs a fresh lineage.
- **Outside the packages.** Root, upgrade, treasury and claim-signer key custody; the off-chain reward manifests (I-3); the SDK and admin console; blast-infra, whose `origin/main` decoders (`7c52521`, 4 November) still target the contracts #168 layouts and not the #198–#212 moves of `ProtocolFeesPaid` to `blast_core::blast_core_events`, the vault events to `blast_vault::blast_vault_events` and the venue events to `blast_migrators::events`. That gap is filed as blast-infra#126; this report makes no statement about replay completeness downstream.
- **Deny lists and pauses of regulated Quotes**, a Cetus pool-manager override, and venue version bumps are accepted trust boundaries (ADR 0134 amendments) and were re-read, not re-rated: the code at this commit does not make any of them worse than the ADR describes.
- **Budget.** All `sui move test` runs used one thread inside the heavy-slot guard; three PoC runs were deferred once by another chat's memory pressure and completed on retry. No coverage report was generated.

## Methodology

**Review.** Each of the 15,112 production lines of the six packages was read line by line in dependency order (admin, core, vault, launchpad, presale, migrators), against the ADRs (0134 and its amendments, 0137 and its 2026-11-05 amendment, 0088, 0097, 0114, 0117), the two prior reviews (Asymptotic October 2026 at `01d76596`; the 3 November pre-audit at `b841a81` and its fixes in #167/#168) and the PR150 reviews. Every prior fix was re-located at this commit before anything else was reported; accepted residuals were re-read for regressions and are not re-reported.

**System map.** Every shared, owned, capability, hot-potato and witness type was tabulated with its abilities and lifecycle; SUI, Quote and Meme were traced from entry to every exit; the invariants each package must hold were written down before the hunt (system-map.md).

**Inventory.** Every `public`, `public(package)` and `entry` function (593) was extracted mechanically with its version gate, pause gate, authority parameter, value flow, emitted events and rounding primitives, then checked by hand (function-inventory.md).

**State tables.** The launch, presale, vault round slot, delayed-primitive and package-version state machines were tabulated function by state and compared with the code.

**Adversarial catalogue.** Every item of the brief's §4 was applied to every package it could apply to; the results, including negative ones, are in checked-clean.md.

**Proof of concept.** A finding is **confirmed** only when a Move test on the audited code demonstrates it (L-1) or when the mechanism is a direct, unconditional reading of the code with no behavioural assumption (I-2, I-3, I-4). I-1 is confirmed by a test of the treasury and gate behaviour. A claim whose trigger could not be executed is a **Hypothesis** and is listed separately. Properties the review relied on were also turned into tests (apportionment, capped crossing, round trips, square root, Merkle negatives), all of which pass.

**Gates.** The repository's style, error-registry, administration-boundary and source-boundary gates and the live ABI gate were run on the snapshot; all pass.

## Commits

| Revision | Commit |
|---|---|
| Audited commit (`origin/main`, PR #212) | `4fe8d739f04bafd0775691795eb9f3727e984089` |
| Test logs and PoCs | same commit, isolated snapshot with CI fixture addresses |
| Paused deep-audit pass (hints only, deleted) | `61e159ce25aaa654f990fd9b340a581aa5283757` |
| Pre-audit (3 November 2026) | `b841a81`; fixes `2de44b6` (#167), `9d768848` (#168) |
| Asymptotic final reviewed commit | `01d7659617abcd552cece6d05b9db409ec4ff2c0` |
| Sui Framework pin | `06734f6ff0af45d8632a14a4dc4b100197f6b1a2` |
| Venue packages verified live | Cetus CLMM v15 `0x260693ec…11f084`, Cetus LP Burn v8 `0xc4aabc87…167cd9`, Bluefin Spot v17 `0xd075338d…486e9c` (5 November 2026) |
| blast-infra decoders read | `7c52521` (`origin/main`, 4 November 2026) |

This report reflects an internal review of the source and local tests at the commit noted above. A review reduces the risk of certain classes of bugs but does not guarantee the absence of all vulnerabilities; only the reviewed packages and properties are covered. It is not a formal proof and not a substitute for the external audit. It is provided for the owner's internal use only, not as financial, legal or investment advice, and no liability is assumed for losses arising from reliance on it.